Tree Decompositions

Heavy-light, centroid, and small-to-large merging

Learning goals

- Prove the **light-edge lemma** and use HLD for path queries in $O(\log^2 n)$
- Handle edge weights and non-commutative aggregates on paths
- Find a **centroid** and build the centroid decomposition in $O(n \log n)$
- Count paths and answer online distance queries with centroids
- Merge per-subtree sets **small-to-large** in $O(n \log n)$ moves

One idea, three tools

Tool	Halving argument
HLD	light edge ⇒ subtree size more than doubles going up
Centroid	components after removal have size ≤ n/2
Small-to-large	moved element's set size at least doubles

Every bound in this lecture is $\mathbf{\log_2 n}$ for the same reason.

Motivating problem — Path Queries II (CSES 2134)

Tree, n, q ≤ $2 \cdot 10^5$. Values on vertices.

- `1 s x` — set value of s to x
- `2 a b` — maximum value on the path a–b

Euler tour handles subtrees, not paths with updates.
Binary lifting handles static paths, not updates.
→ **Heavy-light decomposition.**

Heavy and light edges

Heavy child = child with the largest subtree. Heavy edges form **chains**.



Lemma. Any root→v path has ≤ $\log_2 n$ light edges.
(p, c) light ⇒ size(p) > 2·size(c).

Positions: heavy-first preorder

DFS visiting the heavy child **first**, numbering in preorder:

- each chain = contiguous range
- each subtree = `[pos[v], pos[v] + size[v] − 1]`

One segment tree over positions serves both.

Path query

```
int pathMax(int a, int b) {
    int res = INT_MIN;
    while (head[a] != head[b]) {
        if (dep[head[a]] < dep[head[b]]) swap(a, b);  // deeper HEAD
        res = max(res, segQuery(pos[head[a]], pos[a]));
        a = par[head[a]];
    }
    if (dep[a] > dep[b]) swap(a, b);                  // a = LCA
    return max(res, segQuery(pos[a], pos[b]));
}
```

O(log n) chains × O(log n) per range = **O(log² n)**.

Building HLD (iterative)

```
// pass 1: parents/depths in DFS order, then sizes + heavy child bottom-up
// pass 2: heavy-first preorder
st = {root}; head[root] = root;
while (!st.empty()) {
    int v = st.back(); st.pop_back();
    pos[v] = cur++;
    for (int u : adj[v]) if (u != par[v] && u != heavy[v]) {
        head[u] = u; st.push_back(u);
    }
    if (heavy[v] != -1) {                    // pushed last → popped next
        head[heavy[v]] = head[v]; st.push_back(heavy[v]);
    }
}
```

HLD variations

- **Edge weights:** store w(parent(v), v) at v; last range `[pos[a]+1, pos[b]]`
- **Range updates on paths:** lazy segment tree, same decomposition
- **Non-commutative** (function composition, max subarray):
  two accumulators — u-side upward (reversed), v-side downward
- **LCA for free:** the final `a` in the loop

Centroid

Centroid: removal leaves components of size ≤ n/2.

Finding one: start anywhere; move to a neighbor whose side has size > n/2. Never moves back ⇒ O(n).

```
int findCentroid(int v, int p, int total) {
    for (int u : adj[v])
        if (u != p && !removed[u] && subSz[u] * 2 > total)
            return findCentroid(u, v, total);
    return v;
}
```

Centroid decomposition

```
decompose(component):
    c = centroid(component)
    handle all paths through c          ← O(size)
    remove c; recurse on each piece
```

- Sizes halve ⇒ depth ≤ $\log_2 n + 1$
- Each vertex is in O(log n) components ⇒ **O(n log n)** total

Key property: every path u–v has a unique first-removed centroid c on it, and dist(u, v) = dist(u, c) + dist(c, v).

Counting paths of length k (CSES 2080)

At centroid c, `cnt[d]` = # processed vertices at depth d (`cnt[0] = 1`).

```
for each child subtree T of c:
    for v in T at depth d:  ans += cnt[k - d]     // query first
    for v in T at depth d:  cnt[d]++              // then insert
clear only the touched entries of cnt
```

Query-then-insert per subtree ⇒ endpoints in **different** subtrees.

Total O(n log n).

Online: nearest red vertex (CF 342E)

Per vertex: list of centroid ancestors c with dist(v, c).

Per centroid: `best[c]` = min dist to a red vertex in its component.

```
void paint(int v) { for (auto [c, d] : anc[v]) best[c] = min(best[c], d); }
int query(int v) {
    int res = INT_MAX;
    for (auto [c, d] : anc[v])
        if (best[c] != INT_MAX) res = min(res, best[c] + d);
    return res;
}
```

Both O(log n). Memory O(n log n).

Why the Xenia query is correct

- Every term = length of a walk v → c → red ⇒ **≥ answer**
- Nearest red r; c = first-removed centroid on path v–r:
  c is a common centroid ancestor and
  `best[c] + dist(v, c) ≤ dist(c, r) + dist(v, c) = dist(v, r)` **⇒ ≤ answer**

Pattern: *store something at each centroid ancestor.*

Centroid tree as a structure

CF 321C *Ciel the Commander*: ranks A–Z, equal ranks need a higher rank between.

Centroid-tree depth ≤ $\log_2(10^5) + 1 < 26$ →
rank = letter of the depth in the centroid tree.

Any two same-depth vertices are separated by their common centroid ancestor, which has a smaller depth.

Small-to-large merging

Per vertex: statistic of the multiset of its subtree's values.
Distinct colors (CSES 1139), mode (CF 600E), depth counts (CF 1009F).

- Keep the **largest** child's container (swap: O(1))
- Insert the others' elements into it

Lemma. An element moves only into a set at least as large
⇒ its set size doubles ⇒ ≤ $\log_2 n$ moves.

`std::set` : $O(n \log^2 n)$. Hash/array: $O(n \log n)$.

Small-to-large — code

```
for (int i = n - 1; i >= 0; i--) {          // reverse DFS order
    int v = order[i];
    s[v].insert(color[v]);
    ans[v] = s[v].size();                   // children already merged
    int p = par[v];
    if (p) {
        if (s[p].size() < s[v].size()) swap(s[p], s[v]);
        for (int c : s[v]) s[p].insert(c);
        set<int>().swap(s[v]);              // free memory
    }
}
```

Read the answer **after** children merge, **before** merging upward.

Choosing a tool

Problem	Tool
subtree only	Euler tour + segtree (week 22)
path + updates	HLD
root-path sums, point updates	Euler tour + Fenwick (CSES 1138)
all paths / count / optimize	centroid
online distance to dynamic set	centroid ancestor lists
per-subtree distinct / mode	small-to-large / DSU on tree

Complexity summary

Technique	Build	Per operation
HLD + segtree	$O(n)$	path $O(\log^2 n)$, subtree $O(\log n)$
Centroid decomposition	$O(n \log n)$	$O(\log n)$ ancestor lists
Path counting via centroid	$O(n \log n)$	—
Small-to-large (`set`)	$O(n \log^2 n)$	—

Common mistakes

- HLD: lifting the deeper **vertex** instead of deeper **head**
- Positions not heavy-first → chains not contiguous
- Edge weights: forgetting to skip the LCA
- Centroid: stale sizes; same-subtree pairs counted; `memset` per centroid
- Small-to-large: merging large into small; reading answer too late
- Recursion depth on path-shaped trees → iterative DFS

Practice problems

- HLD: CSES 2134, CSES 1138, SPOJ QTREE, CF 343D
- Centroid: CF 342E, CF 321C, CSES 2080, CSES 2081, IOI 2011 Race, CF 715C
- Small-to-large: CSES 1139, CF 600E, CF 1009F, CF 932F

Summary

- Halving ⇒ $\log_2 n$: light edges, centroid components, merged sets
- HLD: heavy-first preorder, lift deeper head, $O(\log^2 n)$ paths
- Centroid: every path has one "owner" centroid; $O(n \log n)$ total
- Small-to-large: keep the big container, insert the small one
- Next: **Lecture 5 — Flows II** (min-cost flow, Hopcroft–Karp, lower bounds)